설비 고장을 예측하는 AI 데이터 분석

시계열 데이터 구조

날짜·시간 인덱스 만들기와 시간 집계

posco

CONTENTS

목차

01 **날짜 데이터 만들기** 문자열 날짜 · pd.to_datetime · 형식과 오류 · datetime 속성

02 **시간 인덱스 다루기** DatetimeIndex · set_index · sort_index · 부분 문자열 인덱싱

03 **시간 단위 집계** resample · 빈도 문자열 · asfreq · 업샘플링과 보간

04 **실습 — 시간 데이터 다뤄 보기** 변환 · 인덱스 정렬 · 기간 슬라이싱 · 집계 · 업샘플링

posco

01

날짜 데이터 만들기

문자열로 들어온 날짜를 진짜 시간으로 변환하고 조각 추출

posco

문자열 날짜의 문제점

csv에서 막 읽은 날짜는 사실 그냥 글자 덩어리

SYMPTOM · 날짜처럼 보이지만 사실은 글자

"2024-03-01 09:00:00" 같은 날짜가 눈엔 분명 날짜로 보이지만
pandas로 막 읽으면 컴퓨터는 글자가 죽 늘어선 문자열로 인식

글자 상태에서는 시간 계산이 전혀 되지 않음
화면에 날짜처럼 보인다고 컴퓨터도 날짜로 아는 게 아님
읽어 들인 직후엔 반드시 자료형부터 확인

posco

문자열 상태에서는 시간으로 할 수 있는 일이 거의 없음

글자에는 더하기·정렬·자르기가 안 통함

01 · 날짜 계산 불가

3일 뒤 날짜를
구할 수 없음

글자에는 더하기 빼기가 통하지 않으니 "3일 뒤"를 계산 불가

02 · 기간 자르기 불가

시간 순서를
모르는 상태

시작과 끝의 개념이 성립하지 않아 구간 추출 불가

03 · 정렬도 어긋남

03-1과 03-10이
앞뒤 뒤바뀜

글자 순서로 정렬하면 시간 순서와 다르게 줄 서는 일이 흔함

엑셀에서 날짜가 글자로 인식돼 정렬이 이상하던 경험과 똑같은 문제

posco

문자열 날짜와 진짜 시간 비교

글자에서 시간으로 — 변환 한 번에 막혀 있던 모든 기능이 열림

구분	❌ 문자열	✅ 변환 후
인식	글자 덩어리	시간 값
날짜 계산	불가능	가능
기간 자르기	불가능	가능
시간 정렬	어긋남	정확

시간 분석의 첫 단계는 항상 "날짜처럼 보이는 글자"를 진짜 시간으로 바꾸는 일

posco

pd.to_datetime — 시간 분석의 관문

날짜처럼 생긴 글자를 진짜 시간 값으로 바꿔주는 한 줄

GATEWAY · 모든 시간 기능의 출발점

날짜가 담긴 열을 to_datetime에 넣으면
그 열이 통째로 진짜 시간 값으로 변환

이 변환을 거쳐야 모든 시간 기능이 작동
변환하지 않으면 뒤에 배울 인덱스·집계·기간 자르기가 전부 막힘
시간 분석의 출발점 — 이 한 줄이 모든 것의 시작

to_datetime은 단순 형식 변환이 아니라 데이터에 "시간"이라는 능력을 부여하는 작업

posco

변환 전후 자료형 변화

변환이 잘 됐는지는 자료형(dtype)으로 확인

BEFORE · 변환 전

object

글자 상태

컴퓨터는 단순 문자열로 인식
더하기·자르기·정렬 어느 것도 동작하지 않음

AFTER · 변환 후

datetime64

시간 전용 자료형

하이픈·슬래시·시각 포함 등 웬만한 형식 자동 인식
대부분 열 이름만 넣으면 알아서 변환

df['timestamp'].dtype 출력이 datetime64로 나오면 컴퓨터가 시간으로 이해한다는 신호

posco

to_datetime 기본 코드

변환 전후 dtype이 바뀌는지 확인

CODE

```
df = pd.read_csv('engine01_timestamp_sample.csv')
print(df['timestamp'].dtype)
df['timestamp'] = pd.to_datetime(df['timestamp'])
print(df['timestamp'].dtype)

# 출력: object → datetime64[ns]
```

to_datetime은 열 전체를 한 번에 변환 · 수만 개 날짜도 한 줄로 변환되고 같은 자리에 다시 담음

단순한 형식 변환이 아닌 데이터에 "시간"이라는 능력 부여

posco

날짜 형식과 변환 오류

형식이 특이하면 변환 실패 — 이때 형식을 직접 지정

01 · 모호한 형식

03-04-2024

3월 4일? 4월 3일?

국가·시스템마다 표기 순서가 달라 자동 인식이 흔들리는 대표 사례

02 · format 옵션

```
format=
"%Y-%m-%d"
```

연-월-일 순서를 명시하면 해결 — 변환 후 첫 몇 줄로 월·일 확인 필수

03 · errors 옵션

```
errors=
"coerce"
```

빈 값이나 엉뚱한 글자는 errors 옵션으로 NaT 처리

오류 발생 시 당황하지 말고 형식부터 확인 → format으로 순서 알려주기

날짜 형식 기호 정리

연도는 대문자 Y, 월 소문자 m, 일 소문자 d, 시 대문자 H

기호	의미	예시
%Y	연도 4자리	
%m	월 2자리	
%d	일 2자리	
%H	시 2자리	

대부분 to_datetime이 알아서 처리 — 안 되는 경우에만 format 지정

posco

datetime 속성 추출 — .dt로 시간 조각 꺼내기

.dt는 변환된 열에서만 작동 — 변환 먼저, 추출 다음

.dt.year

연도

연도별·계절별 비교의 토대

.dt.month

월

월별로 묶어 계절성 분석 가능

.dt.day

일

일자 기준 분석에 사용

.dt.hour

시

시간대별 운영 패턴 분석의 핵심

조각을 기준으로 시간대별·요일별·계절별 분석 가능 — dt는 datetime의 줄임말

posco

.dt 속성 추출 코드

변환된 시간 열에서 조각 꺼내기

CODE

```
ts = pd.to_datetime(df['timestamp'])
print(ts.dt.year.head()) # 2024
print(ts.dt.month.head()) # 3
print(ts.dt.hour.head()) # 0
```

분은 dt.minute, 요일은 dt.dayofweek · 요일 0은 월요일, 6은 일요일

설비 작동 패턴이 시간대·요일에 따라 다른 경우가 많아 조각 추출이 유용

posco

cycle을 timestamp로 보는 이유

C-MAPSS는 cycle 데이터인데 왜 날짜를 다루는가

WHY · 가상 시각으로 시간 도구 연습

강사가 1 cycle = 1시간으로 약속
cycle 번호를 가상의 시각으로 변환해 샘플 사용

실제 현장 설비 데이터의 상당수가 timestamp 기반
cycle 데이터만으로는 시간 집계를 연습할 수 없음
단, 이 날짜는 진짜 측정 시각이 아닌 연습용 가상 시각

작동 간격이 일정하면 "몇 번째 작동"을 "몇 시간째"로 바꿔 시간 도구 활용 가능

posco

cycle을 시각으로 바꾸는 흐름

137번째 cycle은 "작동 후 137시간째"가 되어 시간 도구 사용 가능

STEP 1

cycle 번호

원본 데이터의 작동 회차 정보

→

STEP 2

시간 약속

1 cycle = 1시간으로 약속

→

STEP 3

가상 시각

timestamp 자료형으로 변환된 시각

→

STEP 4

시간 도구

set_index · resample 등 시간 분석 도구 작동

실제 timestamp 데이터를 만나도 to_datetime → set_index → resample의 흐름이 동일하게 적용

posco

02

시간 인덱스 다루기

DatetimeIndex로 행을 고르고 기간을 잘라내는 법

posco

인덱스 개념과 DatetimeIndex

인덱스는 각 행에 붙은 이름표 — 이 이름표를 시각으로 만든 것이 DatetimeIndex

DEFAULT · 기본 인덱스

0, 1, 2, 3 …

단순 순번

CSV를 막 읽으면 기본으로 부여되는 번호
데이터프레임 맨 왼쪽 숫자가 이 기본 인덱스

DATETIME · 시간 인덱스

2024-03-01 00:00

실제 시각이 이름표

시각을 청구기호 삼아 행을 빠르게 찾아냄
pandas가 시계열로 인식해 시간 기능 활성화

도서관 책의 청구기호처럼 — 인덱스가 행을 찾아내는 청구기호 역할

posco

시간 인덱스로 가능해지는 것

시각이 이름표가 되는 순간 열리는 세 가지 마법

01 · 행 선택

시각으로
원하는 시점 추출

시각만 알면 그 행을 .loc로 바로 골라냄

02 · 시간 집계

하루 단위로 묶어
평균 내기

resample로 시간 단위 평균·최댓값 한 줄 계산

03 · 기간 자르기

3월 1일~3월 3일
쉽게 추출

콜론으로 잇기만 하면 사이 구간 통째로 슬라이싱

posco

기본 인덱스와 시간 인덱스 비교

시간 열을 인덱스로 올리는 순간 pandas의 모든 시간 기능이 열림

구분	❌ 기본 인덱스	✅ 시간 인덱스
이름표	0, 1, 2 순번	실제 시각
행 선택	번호로	시각으로
시간 집계	불가능	가능
기간 자르기	불가능	가능

시간 열을 그냥 두면 평범한 열 — 인덱스로 올리는 순간 시계열로 격상

posco

set_index로 시간 인덱스 만들기

특정 열을 인덱스 이름표로 끌어올리는 도구 — 한 줄이면 충분

USAGE · 인덱스로 삼을 열 이름을 넣기

```
df = df.set_index('timestamp')
```

초보자가 놓치기 쉬운 두 가지 주의

주의 1 — to_datetime으로 변환 먼저, set_index 다음
주의 2 — 결과를 반드시 df에 다시 담기 (`df =`)

문자열 상태로 인덱스를 만들면 시간 기능이 작동하지 않음

변환 먼저, 그다음 set_index — 결과를 반드시 df에 다시 담는 습관

posco

set_index 사용 코드

변환 후 인덱스로 올리고 종류 확인

CODE

```
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.set_index('timestamp')
print(type(df.index).__name__)

# 출력: DatetimeIndex
```

timestamp 열이 맨 왼쪽 이름표 자리로 이동 · 다시 열로 되돌리려면 reset_index

결과를 등호로 다시 df에 담는 게 핵심

posco

sort_index가 필요한 이유

변환·인덱스·정렬 — 시간 분석의 준비 운동 세 박자

01 · 정렬 전제

pandas는 시각 순
정렬을 전제

시각 순서로 정렬돼야 기간 자르기가 작동

02 · 흔한 상황

파일 합치면
순서가 뒤섞임

여러 CSV를 concat하면 시간 순서가 어긋나는 일이 흔함

03 · 정렬 누락

기간 자르기에서
오류·빈 결과

"시작과 끝 사이"라는 개념이 성립 못 해 오류

정렬은 한 번만 해 두면 이후 모든 분석에서 효과 유지

posco

시간 분석 준비 3단계

to_datetime → set_index → sort_index → 준비 완료

STEP 1 · 변환

pd.to_datetime()

문자열 날짜를 datetime64로 변환

→

STEP 2 · 인덱스

set_index()

timestamp 열을 인덱스로 끌어올리기

→

STEP 3 · 정렬

sort_index()

시각 오름차순으로 줄 세우기

→

STEP 4 · 완료

준비 완료

.loc 슬라이싱과 resample 모두 작동

보통 변환·인덱스·정렬을 한 줄씩 이어 쓰고 마지막에 df에 담음 — 한 번만 해 두면 이후 분석 내내 유지

시간 인덱스로 행 선택 — .loc에 시각 넣기

행 번호 대신 시각을 이름표로 부르기 — 조건 필터링보다 짧고 직관적

USAGE · .loc에 시각 넣기

```
df.loc['2024-03-01 09:00:00']
```

시간 인덱스가 있으면 .loc에 시각을 넣어 행을 바로 선택
시간이 이름표가 됐기 때문에 정확히 그 시각의 행이 골라짐
시각의 일부만 넣어도 됨 — 다음 섹션에서 자세히

시각이 인덱스가 되는 순간 .loc로 행을 "직접 부르는" 자연스러운 문법 성립

.loc 행 선택 코드

시각 또는 날짜로 행 선택 — pandas가 시각으로 해석

CODE

```
df = df.set_index('timestamp').sort_index()
print(df.loc['2024-03-01 00:00:00'])
print(df.loc['2024-03-01'].shape)

# 두 번째 출력 shape: (22, 5) — 하루 22개 측정행
```

특정 시각의 행을 고르면 온도·압력·진동이 한꺼번에 나옴 · 그 순간 설비 상태 한눈 파악

날짜만 넣으면 그날 전체 — 정확한 시각을 다 적지 않아도 작동

posco

부분 문자열 인덱싱

시각의 일부만 넣으면 그 기간 전체를 한 번에 선택

01 · 날짜만

```
.loc['2024-03-01']
```

그날 전체

날짜만 적으면 그날 측정값이 통째로 선택됨

02 · 월만

```
.loc['2024-03']
```

그달 전체

월만 적으면 한 달치 모든 측정행이 선택됨

03 · 콜론

```
['2024-03-01' : '2024-03-03']
```

기간 슬라이싱

끝 날짜도 포함 · "고장 직전 일주일"에 딱 맞음

posco

부분 문자열 인덱싱 패턴 정리

시각 전체·날짜만·월만·콜론 — 네 가지 적는 법이 시간 인덱싱의 전부

적는 법	범위	예시
시각 전체	한 행	`2024-03-01 09:00:00`
날짜만	그날 전체	`2024-03-01`
월만	그달 전체	`2024-03`
콜론	기간 전체	`'2024-03-01':'2024-03-03'`

03

시간 단위 집계

resample로 묶고 집계 함수로 추세 드러내기

posco

resample — 시간 단위를 다시 묶는 도구

시간별·일별 평균 같은 집계를 한 줄로 — 시간 인덱스가 있어야 작동

DEFINITION · 다시 표본을 만들기

1시간 데이터를 하루·6시간 단위로 묶기
이름은 어렵지만 결국 "시간 간격을 바꾸는 일"

resample은 시간 인덱스가 있어야 작동 → 변환·인덱스·정렬 준비가 먼저
매일 잰 몸무게를 "주 단위 평균"으로 보면 흐름이 더 잘 보임
이후 학습에서 가장 강력한 도구 — 한 줄이면 분석 절반 완료

posco

다운샘플링과 업샘플링

설비 분석에서는 다운샘플링이 훨씬 자주 쓰임

DOWNSAMPLING · 다운샘플링

촘촘한 데이터를
큰 단위로 묶어 성기게

1시간 → 1일

24개 시간값이 1개 일별값으로 합쳐짐
잡음이 평균으로 사라지고 큰 흐름만 남음
데이터 양 줄어듦 · 결측 발생 없음

UPSAMPLING · 업샘플링

성긴 데이터를
작은 단위로 쪼개 촘촘하게

1시간 → 30분

없던 시점이 새로 생겨 결측이 됨
결측을 어떻게 채울지 따로 정해야 함
데이터 양 늘어남 · 결측 발생

데이터가 충분히 촘촘해서 큰 흐름을 보기 위해 묶는 경우가 많음

posco

두 방향 비교

다운샘플링은 묶어서 성기게, 업샘플링은 쪼개서 촘촘하게

구분	다운샘플링	업샘플링
방향	큰 단위로 묶기	작은 단위로 쪼개기
데이터 양	줄어듦	늘어남
결측	없음	생김
예시	1시간 → 1일	1시간 → 30분

resample을 쓰려면 어느 단위로 묶을지 글자로 알려줘야 함 → 빈도 문자열로 이동

빈도 문자열 이해

resample에 묶을 단위를 약속된 글자로 알려주기

01 · 단위 글자

h · 시간 (소문자)
D · 일 (대문자)
min · 분

시간보다 작은 단위는 소문자, 날짜 이상은 대문자

02 · 숫자 배수

6h · 6시간
15min · 15분
3D · 3일

숫자를 앞에 붙이면 그 배수 단위

03 · 최신 표기 주의

H → h
T → min

인터넷의 대문자 H는 소문자 h로 바꿔 쓰기

예전엔 시간 대문자 H, 분 대문자 T였으나 지금은 소문자 h, min이 표준

posco

자주 쓰는 빈도 문자열

이 빈도 문자열은 시간 데이터를 만들 때도 같은 글자를 사용

단위	글자	예시
시간	`h`	`resample('h')`
일	`D`	`resample('D')`
분	`min`	`resample('min')`
6시간	`6h`	`resample('6h')`

단위를 정했으면 그다음은 어떻게 대표값을 낼지 → 집계 함수

posco

resample 집계 함수

묶기만 해서는 아무 일도 안 일어남 — 집계 함수를 반드시 뒤에 붙이기

01 · MEAN

.mean()

평균 — 추세

그 시간 동안 일반적인 수준을 보여줌 — 추세 관찰에 적합

02 · MAX

.max()

최댓값 — 안전 점검

위험하게 치솟은 순간 — 평균만 보면 묻혀 사라짐

03 · MIN / SUM

.min() .sum()

최솟값·합계

목적에 따라 선택 — 여러 집계 한 번에 구하는 agg도 존재

설비 분석에선 보통 평균으로 흐름 보고 최댓값으로 위험한 순간 점검

posco

집계 함수 사용 코드

일별 평균과 최고 온도 — resample · 점 · 집계 함수

CODE

```
df = df.set_index('timestamp').sort_index()
print(df['temp_core'].resample('D').mean().head(3))
print(df['temp_core'].resample('D').max().head(1))

# 일별 평균: 03-01 ≈ 1402.65, 03-02 ≈ 1403.83, 03-03 ≈ 1405.56
# 첫날 최고 온도: 1410.92
```

쓰는 형태는 항상 동일 · resample로 단위, 점, 집계 함수 — 한 번에 여러 집계는 agg

평균으로 흐름 보고 최댓값으로 위험한 순간 점검 — 설비 분석의 두 시선

posco

asfreq와 resample의 차이

resample은 요약하는 도구, asfreq는 재배치하는 도구

RESAMPLE · 묶어서 집계

```
.resample('D').mean()
```

여러 값을 합쳐 평균·최대

시간 간격 안의 여러 값을 하나의 대표값으로 요약
평균·최댓값이 필요한 일반 집계에 사용

ASFREQ · 간격만 변경

```
.asfreq('D')
```

해당 시점 값만 그대로

합치지 않고 해당 시점 값만 그대로 가져옴
없는 시점은 결측으로 둠 — 누락 시점 확인에 유용

평소 집계엔 resample, 누락 점검엔 asfreq

posco

resample과 asfreq 비교

측정이 누락된 데이터에서 어느 시점이 빠졌는지 점검할 땐 asfreq가 도움

구분	resample	asfreq
동작	묶어 집계	간격만 변경
집계 함수	필요	불필요
누락 시점	집계 처리	결측 표시
주 용도	평균·최대	누락 확인

평소 집계 분석은 resample 주력 · asfreq는 누락 점검의 보조

업샘플링과 결측 발생

업샘플링하면 반드시 결측 발생 — 채우는 방식이 곧 분석 결정

01 · 결측 발생

없던 시점이 빈 값으로 생성

업샘플링의 필연적 부작용 — 결측은 자동으로 채워지지 않음

02 · 앞 채우기 (ffill)

바로 앞 값을 그대로 끌어다 채움

상태가 천천히 변하는 값일 때 빠르고 안전

03 · 보간 (interpolate)

앞뒤 값 사이를 중간값으로 메움

0시 10, 1시 20 → 0시 30분 15 · 온도처럼 부드러운 값에 적합

보간은 추정 — 너무 많은 구간을 채우면 실제와 멀어질 수 있음

posco

업샘플링 후 보간 코드

30분 업샘플링 후 보간 — 결측 201개 → 0개

CODE

```
up = df['temp_core'].resample('30min').mean()
print(up.isna().sum()) # 201
up_filled = up.interpolate()
print(up_filled.isna().sum()) # 0
```

보간 방식 · ffill(앞 채우기) · bfill(뒤 채우기) · interpolate(선형 보간) — 시계열엔 선형 보간이 자연스러움

보간한 값과 실제 측정값은 구분해서 다루는 게 좋음

posco

시간 집계의 분석 의미

시간 집계가 설비 분석에서 가지는 세 가지 효과

01 · 잡음 제거

잔잔한 흔들림
평균으로 상쇄

평균으로 묶으면 잔잔한 흔들림이 사라지고 큰 추세만 남음

02 · 시간대 비교

시간대·요일별
운영 패턴 분석

timestamp 데이터만의 특권 — cycle 데이터로는 불가능

03 · 데이터 경량화

큰 단위로 묶어
가볍게, 흐름 유지

행 수가 줄어 다루기 쉬워지면서 큰 흐름은 보존

너무 큰 단위로 묶으면 위험 스파이크가 평균에 묻힘 — 최댓값도 함께 보기

시간 집계의 세 가지 가치

원본에서는 안 보이던 추세가 일별 평균에서는 또렷하게

VALUE 01

잡음 제거

우연한 흔들림을 걸러 큰 흐름을 드러냄

VALUE 02

시간대 비교

시간대별·요일별 패턴을 나란히 비교

VALUE 03

데이터 경량화

행 수를 줄이면서 큰 흐름은 보존

통계의 "평균이 우연한 흔들림을 줄여 준다"는 원리가 시계열에 그대로 적용

집계 단위 선택 기준

보고 싶은 변화의 시간 규모에 맞춰 단위 선택

PRINCIPLE · 사진 거리와 같은 원리

짧은 변화는 작은 단위, 긴 흐름은 큰 단위
여러 단위로 그려 보고 흐름이 잘 보이는 것 선택

너무 작으면 잡음이 그대로 남아 흐름이 안 보임
너무 크면 의미 있는 변화가 평균에 묻혀 사라짐
한 달치를 분 단위로 묶으면 흐름이 안 보이고, 월 단위로 묶으면 점이 하나뿐

사진을 가까이서 찍느냐 멀리서 찍느냐와 같은 선택

posco

목적별 집계 단위

한 단위만 고집하지 말기 — 큰 단위로 전체 잡고 작은 단위로 확대

보고 싶은 것	단위	이유
하루 내 변화	시간 (h)	시간대 구분 — 24시간 안의 운영 패턴
며칠 추세	일 (D)	흐름 부각 — 일별 평균이 추세를 또렷하게
장기 추세	주 (W)	큰 그림 — 주 단위 평균으로 장기 변화 관찰
순간 이상	분 (min)	세밀 관찰 — 짧은 스파이크·이상 신호 포착

지도에서 전체를 보다 관심 지역을 확대하듯 — 다양한 단위 조합해서 보기

posco

집계 결과 그래프로 보기

집계 결과도 시간 인덱스를 가져 plt.plot으로 그릴 수 있음

01 · 그래프 그리기

숫자로만 보면
흐름이 안 들어옴

그래프로 그려야 흐름이 한눈에 들어옴

02 · 원본과 겹쳐

원본 옅게,
집계 결과 진하게

출렁이는 원본 사이로 매끄러운 추세선이 지나감

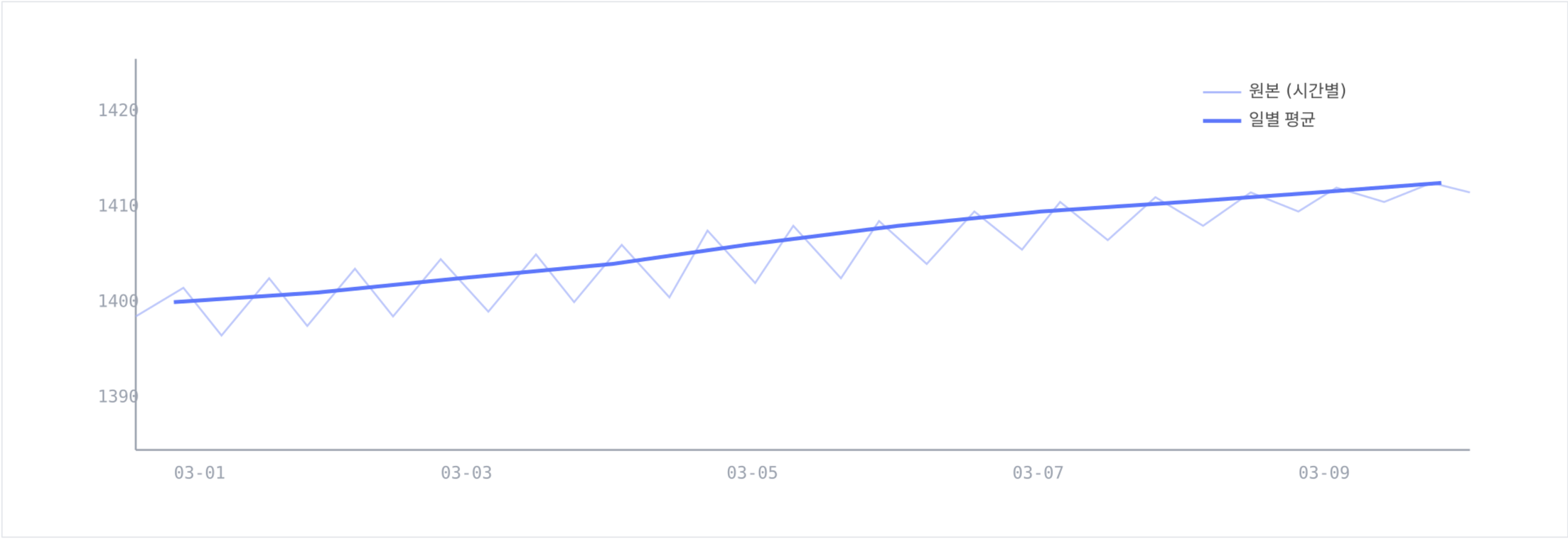
03 · 범례 표시

범례로 두 선
또렷이 구분

label=hourly · label=daily 같은 범례 잊지 말기

원본과 집계 비교 그래프

원본 잡음 위로 매끄러운 일별 평균선이 지나가는 모습



잘 그린 집계 그래프 한 장은 수십 줄의 숫자 표보다 강력

posco

시간 분석 종합 정리

준비·선택·집계 — 시간 분석의 뼈대 세 단계

STEP 1 · 준비

변환 · 인덱스 · 정렬

```
pd.to_datetime()
.set_index()
.sort_index()
```

시계열의 토대 만들기

STEP 2 · 선택

.loc 기간 슬라이싱

```
.loc['2024-03-01']
.loc['03-01' : '03-03']
```

원하는 시점이나 기간 골라내기

STEP 3 · 집계

resample + 집계 함수

```
.resample('D').mean()
.resample('6h').max()
```

묶고 대표값으로 흐름 드러내기

어떤 timestamp 데이터를 만나도 이 순서대로 접근하면 됨

posco

시간 분석 전체 흐름

발전소·공장·건물 에너지 — 어떤 timestamp 데이터에도 동일하게 적용

STEP 1
준비
변환 · 인덱스 · 정렬

→

STEP 2
선택
.loc로 기간 자르기

→

STEP 3
집계
resample + 평균·최대

→

STEP 4
시각화
원본·집계 겹쳐 그리기

다음 시간에서는 cycle 데이터로 돌아가 이동평균·변화량·상관관계 다룸 — 평생 쓰는 기본기

posco

실습 1. 문자열 날짜를 시간으로 변환

> 글자로 읽힌 날짜를 진짜 시간 값으로 바꾸기

목표

글자로 읽힌 날짜를 진짜 시간 값으로 바꾸기

단계

· 샘플을 열고 시간 열의 자료형을 먼저 확인

· 시간 변환 함수로 글자를 시간 값으로 바꾸기

· 바뀐 자료형을 다시 확인해 변환 성공을 확인

예상 결과

자료형이 글자에서 시간 형식으로 바뀜

posco

실습 1. 문자열 날짜를 시간으로 변환 정답

글자로 읽힌 날짜를 진짜 시간 값으로 바꾸기

CODE · 실습 1

```
import pandas as pd
df = pd.read_csv('engine01_timestamp_sample.csv')
print(df['timestamp'].dtype)                  # object (글자)
df['timestamp'] = pd.to_datetime(df['timestamp'])
print(df['timestamp'].dtype)                  # datetime64[us]
```

실습 2. 시간 조각 추출과 파생 열

변환한 시간 열에서 연·월·일·시를 꺼내고 새 열 만들기

목표

변환한 시간 열에서 연·월·일·시 조각을 꺼내고 파생 열 만들기

단계

· 시간 접근자로 연·월·일·시를 각각 꺼내 첫 값 확인

· 시 조각을 새 열로 추가하기

· 새 열의 앞부분을 확인해 시간대별 분석을 준비

예상 결과

첫 행이 2024년 3월 1일 0시, 시 열 앞 세 값은 0·1·2

posco

실습 2. 시간 조각 추출과 파생 열 정답

변환한 시간 열에서 연·월·일·시를 꺼내고 새 열 만들기

CODE · 실습 2

```
print(df['timestamp'].dt.year.iloc[0], df['timestamp'].dt.month.iloc[0])
print(df['timestamp'].dt.day.iloc[0], df['timestamp'].dt.hour.iloc[0])
# 2024 3 / 1 0
df['hour'] = df['timestamp'].dt.hour
print(df['hour'].head(3).tolist())          # [0, 1, 2]
```

실습 3. 시간 인덱스 만들고 정렬

시간 열을 인덱스로 올리고 정렬해 분석 준비 완료

목표

시간 열을 인덱스로 올리고 정렬해 시간 분석을 준비

단계

· 시간 열을 인덱스로 끌어올려 결과를 다시 담기

· 인덱스 종류를 확인해 시간 인덱스인지 보기

· 정렬 뒤 첫·마지막 시각과 전체 행 수 확인

예상 결과

시간 인덱스로 바뀌고 3월 1일 0시~8일 23시, 182행

posco

실습 3. 시간 인덱스 만들고 정렬 정답

시간 열을 인덱스로 올리고 정렬해 분석 준비 완료

CODE · 실습 3

```
df = df.set_index('timestamp').sort_index()
print(type(df.index).__name__)               # DatetimeIndex
print(df.index[0], '~', df.index[-1])
# 2024-03-01 00:00:00 ~ 2024-03-08 23:00:00
print('전체 행:', len(df))                    # 182
```

posco

실습 4. 시간 인덱스로 기간 잘라보기

특정 날짜와 기간을 시간 인덱스로 골라내기

목표

시간 인덱스로 특정 날짜와 기간을 골라내기

단계

· 하루 날짜를 넣어 그 날 데이터만 선택

· 시작과 끝 날짜를 콜론으로 이어 기간 선택

· 각 결과의 행 수로 기간 크기 확인

예상 결과

3월 2일 하루 20행, 3월 5일~7일 사흘 70행(끝 포함)

posco

실습 4. 시간 인덱스로 기간 잘라보기 정답

특정 날짜와 기간을 시간 인덱스로 골라내기

CODE · 실습 4

```
day = df.loc['2024-03-02']
print('3월 2일 행:', len(day))                # 20
span = df.loc['2024-03-05':'2024-03-07']
print('사흘 행:', len(span))                   # 70 (끝 날짜 포함)
```

posco

실습 5. 시간 단위 평균·최댓값 집계

리샘플링으로 단위를 바꿔 대표값 계산

목표

시간 단위를 바꿔 묶고 평균·최댓값 대표값을 계산

단계

· 6시간 단위로 묶어 평균을 구하고 앞부분 확인

· 같은 단위로 최댓값을 구해 평균과 비교

· 최댓값이 평균보다 큼을 수치로 확인

예상 결과

6시간 첫 구간 평균 약 641.06, 최댓값 641.18

posco

실습 5. 시간 단위 평균·최댓값 집계 정답

리샘플링으로 단위를 바꿔 대표값 계산

CODE · 실습 5

```
print(df['temp_out'].resample('6h').mean().head(2))
# 641.058 / 641.005
m = df['temp_out'].resample('6h').max().iloc[0]
print('6h 최댓값 첫 구간:', round(m, 2))       # 641.18
```

실습 6. 리샘플링 단위와 데이터 성김

묶는 단위를 바꿔 다운샘플링 효과를 체험

목표

묶는 단위를 바꿔 데이터가 성겨지는 다운샘플링을 체험

단계

· 한 시간·6시간·12시간 단위로 각각 묶어 평균 구하기

· 각 결과의 행 수를 세어 비교

· 단위가 클수록 행 수가 줄어 흐름이 매끄러워짐을 확인

예상 결과

한 시간 192행, 6시간 32행, 12시간 16행으로 줄어듦

posco

실습 6. 리샘플링 단위와 데이터 성김 정답

묶는 단위를 바꿔 다운샘플링 효과를 체험

CODE · 실습 6

```
for f in ['h', '6h', '12h']:
    n = len(df['temp_out'].resample(f).mean())
    print(f, '단위 행:', n)
# h 192,  6h 32,  12h 16   (단위가 클수록 성겨짐)
```

실습 7. 일별 집계로 추세 확인

일별 평균으로 잡음을 걸러 추세를 원본과 겹쳐 확인

목표

일별 평균으로 잡음을 걸러 낸 추세를 원본과 겹쳐 확인

단계

· 하루 단위 평균을 구하고 앞부분 확인

· 원본을 옅게, 일별 평균을 진하게 겹쳐 그리기

· 첫날과 마지막날 평균을 비교해 추세 방향 판정

예상 결과

일별 평균 641.07에서 642.35로 상승, 매끄러운 추세선

posco

실습 7. 일별 집계로 추세 확인 정답

일별 평균으로 잡음을 걸러 추세를 원본과 겹쳐 확인

CODE · 실습 7

```
import matplotlib.pyplot as plt
daily = df['temp_out'].resample('D').mean()
plt.plot(df.index, df['temp_out'], alpha=0.4, label='hourly')
plt.plot(daily.index, daily.values, linewidth=2, label='daily mean')
plt.legend(); plt.show()
print(round(daily.iloc[0], 2), '->', round(daily.iloc[-1], 2))
# 641.07 -> 642.35  (상승 추세)
```

posco

실습 8. 업샘플링 결측과 보간

촘촘하게 만들 때 생기는 결측과 원본 결측을 보간으로 채우기

목표

업샘플링으로 생기는 결측과 원본의 기존 결측을 보간으로 채우기

단계

· 30분 단위로 촘촘하게 만들어 생긴 결측 세기

· 보간으로 빈 값을 채우고 결측이 0이 됨을 확인

· 원본에 원래 있던 결측도 보간으로 메우기

예상 결과

업샘플링 결측 201이 보간 후 0, 진동 원본 결측 3도 0

posco

실습 8. 업샘플링 결측과 보간 정답

촘촘하게 만들 때 생기는 결측과 원본 결측을 보간으로 채우기

CODE · 실습 8

```
up = df['flow'].resample('30min').mean()
print('업샘플링 결측:', int(up.isna().sum()))          # 201
print('보간 후:', int(up.interpolate().isna().sum()))   # 0
print('진동 원본 결측:', int(df['vibration'].isna().sum()))         # 3
print('진동 보간 후:', int(df['vibration'].interpolate().isna().sum()))  # 0
```